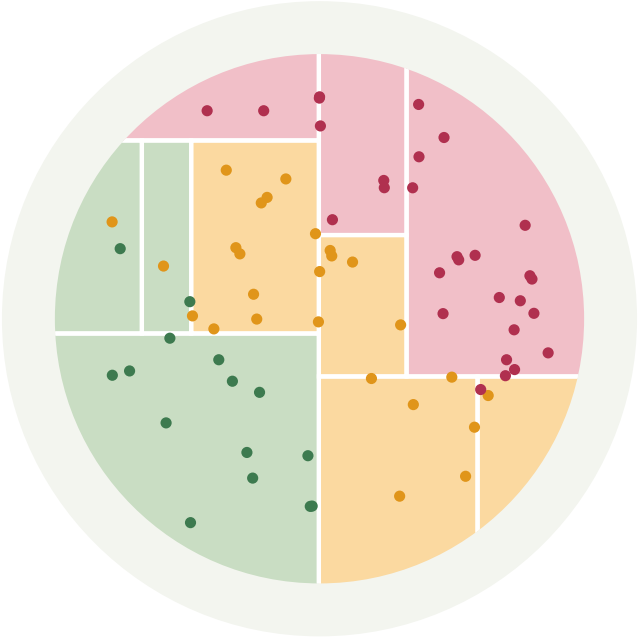
Решающие деревья

Урок

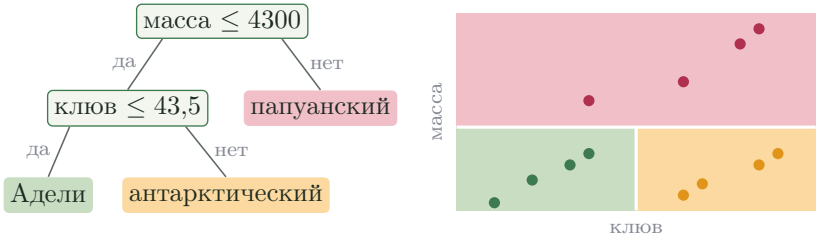


Второй из пяти уроков о моделях. Из урока «Ближайшие соседи» здесь нужны объекты, признаки и классы, train и test, доля верных ответов, переобучение и скользящий контроль. Пример тот же — двенадцать пингвинов трёх видов: Адели (класс 0), антарктический (1) и папуанский (2), признаки — длина клюва в миллиметрах и масса в граммах, а для регрессии — ещё длина крыла. Ячейка задаёт их заново:

```
import numpy as np
# клюв, мм; масса, г
X = np.array([[38, 3600], [40, 3800], [36, 3300],
              [41, 3950], [47, 3550], [50, 3800],
              [46, 3400], [51, 3950], [46, 4900],
              [49, 5400], [41, 4650], [50, 5600]])
y = np.array([0, 0, 0, 0, 1, 1, 1, 1, 2, 2, 2, 2])
# крыло, мм
fl = np.array([185, 190, 182, 195, 193, 198,
               189, 201, 214, 221, 210, 224])
m = X[:, 1]
```

1 Решающие деревья

Решающее дерево задаёт вопросы по одному признаку: «$x_j \le t$?». От ответа зависит следующий вопрос, а в конце пути — *лист* с ответом. Слева — дерево для пингвинов, которое построим ниже, справа — как оно делит плоскость признаков:



Хороший вопрос делит объекты узла так, что в каждой части ответы почти одинаковы. Неоднородность узла меряют *примесью*. Пусть доли классов в узле — $p_0, \dots, p_{K-1}$; *критерий Джини* и *энтропия* —

$$G = 1 - \sum_c p_c^2, \qquad H = -\sum_c p_c \log_2 p_c,$$

слагаемое с $p_c = 0$ считают нулём. Обе равны 0 в чистом узле, где один класс, и наибольшие при равных долях: при K равных классах $G = 1 - 1/K$ и $H = \log_2 K$. Смысл Джини: $\sum_c p_c^2$ — вероятность, что два объекта, взятые из узла наугад с возвращением, одного класса, G — что разных.

```
def gini(y):
    p = np.bincount(y) / len(y)
    return 1 - (p ** 2).sum()

def entropy(y):
    p = np.bincount(y) / len(y)
    p = p[p > 0]
    return -(p * np.log2(p)).sum()

print(gini(y), entropy(y))
print(gini(np.array([1, 1, 1, 2])))
```

```
0.6666666666666667 1.584962500721156
0.375
```

Прирост — насколько вопрос уменьшил примесь: из примеси узла вычитают примеси частей, взвешенные их долями. Если из n объектов узла n_L ушли влево и n_R вправо, то

$$\Delta = I(\text{узел}) - \frac{n_L}{n} I(L) - \frac{n_R}{n} I(R),$$

где I — Джини или энтропия. Лучший вопрос — с наибольшим приростом. Пороги: между двумя соседними значениями признака любой порог делит объекты одинаково, поэтому хватает одного на промежуток, и берут середину — она одинаково далека от обеих сторон. У признака с m

различными значениями $m-1$ порогов. Перебор по обоим признакам на всех двенадцати пингвинах — лучший порог и приросты всех порогов:

```
def gain(x, y, t, crit=gini):
    L, R = y[x <= t], y[x > t]
    n = len(y)
    return (crit(y) - len(L) / n * crit(L)
            - len(R) / n * crit(R))

for j in [0, 1]:
    v = np.unique(X[:, j])
    ts = (v[1:] + v[:-1]) / 2
    g = [gain(X[:, j], y, t) for t in ts]
    print(ts[np.argmax(g)])
    print(np.round(g, 3))
```

```
43.5
[0.061 0.133 0.222 0.248 0.133 0.083 0.074 0.061]
4300.0
[0.061 0.033 0.074 0.083 0.167 0.333 0.222 0.133
 0.061]
```

Пример (дерево для пингвинов). Лучший порог по массе — 4300, середина между 3950 и 4650: справа четыре папуанских, слева остальные восемь, прирост $\frac{2}{3} - \frac{8}{12} \cdot \frac{1}{2} = \frac{1}{3}$. Клюв проигрывает, и корень — «масса ≤ 4300?». Правая часть — чистый лист. В левой по четыре Адели и антарктических, $G = \frac{1}{2}$, и перебор начинается заново, по её восьми объектам. Масса там почти ничего не даёт, у двух видов она похожая, а «клюв $\le 43{,}5$?» делит узел начисто, с приростом $\frac{1}{2}$. Листья чистые, дерево готово: глубина 2, три листа. На тесте из урока «Ближайшие соседи» оно не ошибается: $(41, 3520)$ — лёгкий, клюв короткий, Адели; $(45, 3450)$ — лёгкий, клюв длинный, антарктический; $(44, 4750)$ — тяжёлый, папуанский.

Так строится любое дерево — *жадно*: в корне взять лучший вопрос по всем признакам и порогам, разделить объекты и в каждой части повторить то же, как будто она — вся выборка. Жадно — потому что вопрос выбирают по приросту сейчас, не заглядывая, что он даст ниже. Узел становится листом, когда в нём один класс, когда его не делит никакой вопрос или когда сработал *останов*: достигнута наибольшая глубина, в узле мало объектов, прирост мал. Ответ листа — самый частый класс его объектов, доли классов в листе — оценка вероятностей. Масштаб дереву не важен: вопрос «$x_j \le t$?» после стандартизации — тот же вопрос с другим t.

Дерево регрессии

В регрессии примесь узла — дисперсия ответа в нём, то есть MSE ответа «среднее узла», а ответ листа — среднее его y. Функция `gain` годится и здесь, с `np.var` вместо `gini`. Масса по длине крыла — лучший первый вопрос и ответы двух листов:

```
v = np.unique(fl)
ts = (v[1:] + v[:-1]) / 2
g = [gain(fl, m, t, crit=np.var) for t in ts]
best = ts[np.argmax(g)]
left, right = m[fl <= best], m[fl > best]
print(best, left.mean(), right.mean())
```

```
205.5 3668.75 5137.5
```

Глубина и переобучение

Без останова дерево растёт, пока листья не станут чистыми, и на train не ошибается вовсе, если там нет одинаковых объектов с разными ответами. Глубокие листья держат по одному-два объекта — это уже не закономерность, а запомненный шум; мелкое дерево, наоборот, слишком грубое.

Глубину выбирают скользящим контролем; опыт — в разделе 2.

2 То же в sklearn

Порядок вызовов — как в уроке «Ближайшие соседи»: создать объект с гиперпараметрами, `fit(X, y)`, `predict(X)`.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_text

tree = DecisionTreeClassifier(max_depth=2).fit(X, y)
names = ["клюв", "масса"]
print(export_text(tree, feature_names=names))
```

```
|--- масса <= 4300.00
|   |--- клюв <= 43.50
|   |   |--- class: 0
|   |--- клюв >  43.50
|   |   |--- class: 1
|--- масса >  4300.00
|   |--- class: 2
```

Это дерево из раздела 1. Критерий — `criterion="gini"` по умолчанию или `"entropy"`; останов — `max_depth`, `min_samples_leaf`, `min_impurity_decrease`. Пороги — середины, но признаки sklearn хранит в `float32`, с семью значащими цифрами.

Дерево регрессии — `DecisionTreeRegressor`, примесь — MSE, как у нас:

```
from sklearn.tree import DecisionTreeRegressor

reg = DecisionTreeRegressor(max_depth=1)
reg.fit(fl[:, None], m)
print(reg.tree_.threshold[0])
print(reg.predict([[190], [215]]))
```

```
205.5
[3668.75 5137.5 ]
```

Опыт с глубиной: класс — знак $x_1 + x_2$, у каждой пятой точки перевёрнут; train и test по сто точек, `score` — доля верных:

```
rng = np.random.default_rng(0)
P = rng.normal(size=(200, 2))
c = (P.sum(axis=1) > 0).astype(int)
c[::5] = 1 - c[::5]
for depth in [1, 2, 4, 8, None]:
    tr = DecisionTreeClassifier(max_depth=depth,
                                random_state=0)
    tr.fit(P[:100], c[:100])
    print(depth, tr.score(P[:100], c[:100]),
          tr.score(P[100:], c[100:]))
```

```
1 0.71 0.61
2 0.82 0.67
4 0.83 0.65
8 0.93 0.63
None 1.0 0.58
```

На train чем глубже, тем лучше, до безошибочного дерева без останова; на test — сначала лучше, потом хуже: дерево учит шум.

Ловушка. Ничьи у дерева. Из вопросов с равным приростом sklearn берёт первый встреченный, а признаки

перебирает в случайном порядке, заданном `random_state`; без него дерево на тех же данных бывает разным. Два одинаковых столбца массы — номер признака в корне при зёрнах $0, \ldots, 5$:

```
D2 = np.column_stack([X[:, 1], X[:, 1]])
for seed in range(6):
    tr = DecisionTreeClassifier(max_depth=1,
                                random_state=seed)
    print(tr.fit(D2, y).tree_.feature[0], end=" ")
```

```
1 1 0 0 0 1
```

Дальше — урок «Линейная регрессия»: модель, у которой ответ — формула от признаков, и выучить её — значит найти коэффициенты.